- `MapDelete("/fruit/{id}", (string id) => {/* */})`

Additionally, the last four endpoints need to validate the `id` parameter. This validation can be extracted to a helper method and applied as a filter, but you still need to *remember* to apply the filter when you add a new endpoint.

All this duplication can be removed by using route groups. You can use route groups to extract common path segments or filters to a single location, reducing the duplication in your endpoint definitions. You create a route group by calling `MapGroup("/fruit")` on the `WebApplication` instance, providing a route prefix for the group (`"/fruit"`, in this case), and `MapGroup()` returns a `RouteGroupBuilder`.

When you have a `RouteGroupBuilder`, you can call the same `Map*` extension methods on `RouteGroupBuilder` as you do on `WebApplication`. The only difference is that all the endpoints you define on the group will have the prefix `"/fruit"` applied to each endpoint you define, as shown in figure 5.11. Similarly, you can call `AddEndpointFilter()` on a route group, and all the endpoints on the group will also use the filter.